

Where the platform stands

The content engine, how an analyzed design reaches a client's WordPress site, what is proven by measurement, and what is still blocked.

PREPARED FOR

Zain

DATE

30 September 2026

BRANCH

main · 86d9642

PRODUCTION

Live, healthy

01 — STATUS

At a glance

Every figure here was produced by running the system, not by reading the code.
Where something is unproven, it says so.

7,822

backend tests
passing, 0 failing

484

files type-clean
under mypy

53

of 53 Web 2.0
adapters built

0

pixels differing, local
vs WordPress

AREA	STATE	NOTE
Content engine	BUILT	8 templates, 7 typed slots each, evidence-gated
WordPress publishing	BUILT	Plugin v1.19.0 installed and answering
Web 2.0 module	BUILT	Awaiting platform API credentials
Browser extension	BUILT	v0.2.0, one-click multi-directory fill
Audit engine	BUILT	Runs end to end on the cheapest model tier
AI at runtime	BLOCKED	Anthropic account has no credit
Live WordPress push	UNPROVEN	Host CDN challenges the API path

02 — CONTENTS

What is in here

01	Status at a glance	02
02	Orientation — what AIOS is, and the repo	04
03	The content engine — pages are filled, not written	05
04	Design delivery — the design travels with the page	06
05	Images — provider chosen by measurement	08
06	Cost — every model call on the cheapest tier	09
07	Off-page — Web 2.0 and the extension	10
08	Production — what is deployed	11
09	Verification — proven, and not proven	12
10	Open items — what is blocking what	13
11	Running it locally	14

02 — ORIENTATION

What AIOS is

A multi-tenant SEO platform an agency operates on behalf of its clients. Five modules: **Audit** (site crawl, findings, a client-facing report), **Content** (generate and publish pages to the client's own site), **Off-page** (citations, directories, Web 2.0 placements), **Portal** (what the client logs into) and **Radar** (algorithm and policy monitoring).

Stack

- **Backend** — FastAPI, Celery workers, PostgreSQL 16 with row-level security enforced per tenant, Redis.
- **Frontend** — Next.js dashboard and client portal.
- **Extension** — Chrome MV3 side panel, for work that must happen in a real logged-in browser.
- **WordPress plugin** — "AIOS Publisher", installed per client site, receiving pages over a shared-key REST route.
- **Deploy** — Portainer git stack on a VPS; `main` is the deploy source.

Repository layout

PATH	CONTAINS
<code>backend/</code>	FastAPI app, Celery workers, provider integrations, tests
<code>frontend/</code>	Next.js operator dashboard and client portal
<code>extension/</code>	Chrome MV3 extension — TypeScript, Vite, Vitest
<code>wordpress-plugin/</code>	AIOS Publisher source and built zip
<code>db/migrations/</code>	Numbered SQL migrations, applied in order on deploy
<code>infra/</code> <code>docs/</code>	Compose files and entrypoints; written documentation

03 — THE CONTENT ENGINE

Pages are filled, not written

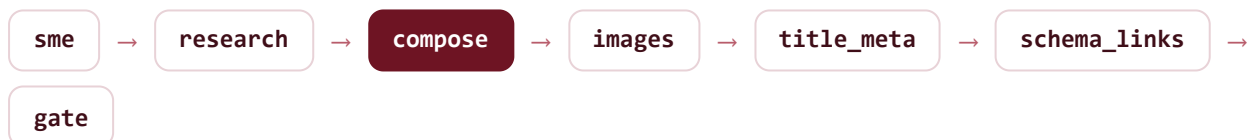
The core design decision, and the one most worth understanding before touching this module.

A page is not free-form prose. Every page type is a **fixed wireframe of exactly seven sections**, and each section is a set of typed slots the model fills. A service page is hero → features → process → FAQ → testimonials → pricing → CTA, in that order, always.

Three rules follow, and they are what keep the output honest:

- **A section with no client evidence is dropped, never invented.** Testimonials and pricing are evidence-gated: no supplied quotes means no testimonial band, rather than a plausible fabricated one.
- **The analyzed design is applied to that wireframe**, so a page looks like the client's own site rather than a template.
- **Generated images are placed into their sections**, not scattered through the prose.

The templated pipeline



Only `compose` writes the page: one governed model call returns JSON keyed by slot id. `research`, `schema_links` and the claims audit are pure Python and cost nothing. Eight templates exist — about, blog, FAQ, homepage, local, location, service, service area — and a longer prose pipeline still serves article-shaped pages.

04 — DESIGN DELIVERY

The design travels with the page

Worth settling early, because the intuitive answer is the wrong one.

When AIOS analyzes a client's site it extracts a design profile — palette, typefaces, button shape. That profile is **not** written into the WordPress theme. Each published page carries its own complete stylesheet, stored as post meta and injected into `<head>` for that post alone.

MEASURED ON A GENERATED PAGE	RESULT
CSS rules total	135
Scoped to <code>.aios-doc</code>	134
Unscoped	1 — a box-sizing reset
<code>body</code> / <code>html</code> styled globally	No
Theme options written	None

Why per-page is right. A theme-level write is global and destructive: a wrong analysis breaks the whole site rather than one page, and two page types could never carry different designs. Per-page works on any theme with no compatibility work, and deleting the post removes the styling with it.

The real gap, for later

The page *body* is ours; the **chrome** — header, nav, footer — is still the client's theme. In practice they agree, because the design profile was analyzed from that same site. It only diverges when publishing to a site whose theme differs from the analyzed source, or for a client with no site yet. Worth a decision eventually; it is not a defect.

04 — DESIGN DELIVERY, CONTINUED

Parity with WordPress

The published page is the local file put through a pipeline: WordPress strips tags it does not recognise, the plugin wraps the body, and the stylesheet is re-sanitised. That pipeline was simulated from the plugin's own source and the result pixel-diffed against the local render. It found two real breakages.

Every inline SVG was being deleted

WordPress's content sanitiser does not allow SVG, so card icons, hero ticks and FAQ chevrons published as empty boxes — on a live client site. The plugin now permits a tight geometry-only subset: no scripts, no external references, no embedded images.

The plugin's own stylesheet was decorating our headings

It added a coloured rule beneath every second-level heading, which the design never drew. It is now skipped entirely for a self-contained page, exactly as it already was for pages built in a page builder. Two stylesheets on one page is the same class of defect as two renderers, expressed in CSS.

With both fixed, the local render and the simulated WordPress render differ by **0 of 1,792,000 pixels**.

One caveat, stated plainly

This is a **simulation**, not a live round-trip. It runs the page through the plugin's actual transformations and is strong enough evidence to have caught two genuine bugs — but a real end-to-end publish has not been completed since the plugin was updated. See section 09.

05 — IMAGES

Provider chosen by measurement

Six model and quality pairs were run against the same prompt, with cost taken from the provider's own token meter rather than from published estimates.

MODEL	QUALITY	COST	TIME	VERDICT
gpt-image-1-mini	medium	\$0.0129	18s	REJECTED
gpt-image-2	low	\$0.0057	14s	Usable, muddy
gpt-image-2	medium	\$0.0421	32s	Good
gpt-image-2	high	\$0.1656	82s	Excellent
gpt-image-2.5-flare	medium	\$0.0113	13s	CHOSEN

The cheapest option was **rejected on quality, not price**: it rendered a visible face and a flat clip-art chart, breaking two hard rules of the photo brief. **flare at medium** matches `gpt-image-2` at high quality for a fifteenth of the price and a sixth of the latency, and held the brief across four validation scenes. It is also deterministic — exactly 343 output tokens every run, where the provider's adaptive setting can silently choose a tier sixteen times dearer.

Page weight, which matters on an SEO product

PNG output was **1.92 MB** per image; five per page is roughly ten megabytes, which fails Core Web Vitals on the very product meant to improve them. Requesting WebP gives **0.12 MB** — a sixteenfold reduction at identical token cost and no visible quality loss.

06 — COST

Every call on the cheapest tier

On 26 September the platform spent **\$9.47 across 74 model calls on 11 pages** — about \$0.86 a page. The cause was not subtle: both model tiers were set to the dearest model available, so the page writer and every utility call ran on it.

Everything now runs on `claude-haiku-4-5`: the writer, heavy folds, the audit engine's agents, policy research, content research and the design reader. That is **5× cheaper** than the tier it replaced.

A live breakage found while deploying this

Production's stack environment pointed the writer at a model id left over from a gateway removed on 25 September. Production now calls the provider directly, where that id does not exist — so **content generation in production was aimed at a model that is not there**, while the audit engine's 21 agents were still on the dearest tier. Both fixed and verified inside the running container.

Carry this forward: a value pinned in the deploy stack's environment beats the compose default, so a setting can go stale there while the code looks correct. Prefer dropping the pin so the git-tracked default governs.

Two model calls that did not need a model

Of roughly 6.7 model calls per page, only two write anything a reader sees. Two of the rest already had working deterministic Python paths, so those were promoted from a fallback to a deliberate switch — the QA judge, and the photo-scene author. Both default on; turned off for bulk runs of a proven page type they take a page from about **\$0.86 to roughly \$0.12**.

07 — OFF-PAGE

Web 2.0 and the extension

Web 2.0: adapters complete

Fifty-four platforms are declared; Medium is excluded by design because its publishing API was withdrawn. Of the 53 remaining, **all 53 have a registered publisher builder** — no gaps, no orphans. Each platform activates the moment its API credential reaches the vault, with no code change. Roughly 19 hold credentials today; the other 34 are built and idle.

Extension v0.2.0: one click, every directory

Directory citation work used to be ten manual rounds per batch — focus a tab, press fill, read the result, move on. The tabs were already open and each already knew which task it belonged to, so the operator was acting as a for-loop.

A **fill-all sweep** now fills every open directory from one button, each in its own background tab, without the operator changing tab. It runs sequentially rather than in parallel — ten content scripts racing is where a tab gets skipped — one directory's failure never aborts the other nine, and the panel reports per-directory outcome rather than a single total.

It fills; it does not submit

Filling is reversible and local. Submitting creates a public listing under a client's name on a third-party site, and a wrong one cannot be recalled — a form that reports as complete can still be wrong in ways only a human sees. So the sweep stops at filled, and submitting stays one-by-one under the operator's eye. The change removes drudgery from the filling side, not judgement from the deciding side.

08 — PRODUCTION

What is deployed

A Portainer git stack on the VPS, deploying from `main`. Seven containers: API, worker, beat, web, Postgres, Redis, and a migrate container that runs first and must exit cleanly before anything else starts.

CHECK	RESULT
Containers	ALL HEALTHY migrate exited 0
Writer, heavy and audit model	<code>claude-haiku-4-5</code>
Image model	<code>gpt-image-2.5-flare</code> , medium
External API	Owner login 200; bad credentials 401
Migrations applied	162

The database reset

Production's stale data was cleared on 27 September: **19,467 rows across 119 tables** removed, 451 kept. A full verified backup was taken first and still exists.

Deliberately kept: the user logins, the two clients those logins belong to, the directory catalogue (226 rows), the Web 2.0 platform catalogue (90), and 96 encrypted Web 2.0 account credentials — those last are logins to accounts that exist on external sites and cannot be recreated from this database.

The migration ledger lives in a separate schema, so clearing the public schema left it intact and the next deploy applied only new migrations rather than replaying all 162.

09 — VERIFICATION

Proven, and not proven

The distinction matters more than the green ticks.

SURFACE	RESULT
Backend tests	7,822 PASSED 4 skipped, 0 failed
Backend lint and types	CLEAN ruff; mypy across 484 files
Extension	131 PASSED TypeScript clean
Frontend	BUILDS CLEAN
Secrets in git	NONE no env files, no live keys
Live WordPress round-trip	NOT PROVEN
Web 2.0 adapters vs live APIs	PARTLY only those with credentials

Two honest limits

The WordPress parity result is a simulation. It runs the page through the plugin's real transformations, taken from the plugin source, and pixel-diffs the result. Strong evidence — it found two genuine bugs — but not a completed live publish.

Every Web 2.0 platform has an adapter; not every adapter has been exercised. Only the platforms holding credentials have ever made a real API call. The rest are structurally complete and untested against a live service.

10 — OPEN ITEMS

What is blocking what

Needs an account owner, not an engineer

1 Anthropic credit

The key is valid and the balance is zero. Content generation, audits and policy all fail until it is topped up. The single biggest blocker — everything else is built and waiting behind it.

2 A seeded owner account keeps returning

The deploy re-creates an owner login from seed variables on every deploy, with a short password, and owner role reaches the client vault and all tenant data. Either strengthen that password or clear those variables *first* and then remove the account — deleting it alone will not hold.

3 Rotate exposed credentials

A source-control token, the deploy API token, a server password and an image-provider key were exposed during this work. Ask Adan for the list; none appear in this document.

4 Host CDN blocks the publishing API

The client site's host intermittently serves a browser challenge on the REST path the platform publishes through. It needs an exemption for that path, or an allowlist entry for the platform server. Intermittent is worse than blocked: a publish interrupted mid-flight can half-apply.

5 Web 2.0 platform API keys

Thirty-four built adapters are idle for want of credentials. Each key added activates that platform immediately, with no code change.

10 — OPEN ITEMS, CONTINUED

Engineering, once unblocked

- Complete a real end-to-end publish to a live WordPress site and confirm it against the parity simulation.
- Regenerate the two existing client pages — their prose predates the readability rule, so they still carry too many long sentences for the SEO plugin's threshold.
- Decide whether the site-chrome gap in section 04 is worth closing.
- Push the model-tier defaults, then drop the corresponding deploy-stack pins so the git-tracked values govern.

Running it locally

Postgres and Redis run locally; the backend needs a virtualenv and an environment file — ask Adan, as it carries live provider keys and is deliberately not in git.

TASK	COMMAND
Backend gate	<code>ruff check . && mypy app workers integrations && pytest -m unit</code>
Extension gate	<code>npx tsc --noEmit && npx vitest run && npm run build</code>
Frontend	<code>npx tsc --noEmit && npm run build</code>

Two practical notes

The full backend suite is 7,822 tests and takes **hours, not minutes**. For a quick signal run ruff and mypy plus a targeted selection; keep the full run for before a release.

Piping pytest through another command **hides its exit code** — read the summary line, not the exit status. That mistake briefly turned a failing run into a reported pass during this work.